

SIMATS ENGINEERING
ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME:	ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS	COURSE CODE:	MLA010 1
---------------------	---	---------------------	---------------------

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100
No. of Questions: 1

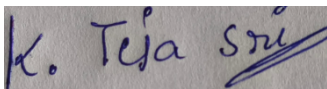
Time: 90 Mins

Annexure A
AI Model Design Exercise

Code of Conduct:

I K.Teja Sri(192525249) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	Smart Loan Approval: A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	Student Performance Prediction: A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	Disease Risk Prediction: A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	Image Recognition System: An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.
5	Handwritten Character Recognition: A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.

6	Non-Linear Classification: A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	Intelligent Recommendation Agent: An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	Autonomous Game Agent: An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	Explainable Diagnosis System: A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	AI Model Selection Challenge: A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	Problem Statement	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	Objectives	State the specific goals of developing the AI model and expected outcomes.
3	Dataset Description	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	Data Preprocessing	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	Selected AI Technique and Justification	Identify the selected AI technique and justify its suitability for the problem.
6	Model Design / Architecture	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	Algorithm / Pseudocode	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	Implementation	Provide programming language, libraries/tools, important code sections, and implementation details.
9	Experimental Results	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	Performance Evaluation	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.
11	Result Analysis	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	Limitations	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	Future Enhancements	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	Conclusion	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	References	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	GitHub Repository Link	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	10%	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	10%	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	10%	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	10%	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	10%	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.

6. Algorithm / Pseudocode	5%	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	15%	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well-organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non-functional or GitHub repository is missing.
8. Experimental Design and Results	10%	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	10%	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.
10. Result Analysis and Interpretation	5%	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.

11. Limitations and Future Enhancements	5%	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	10%	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				

AI Model Design Exercise – Question 1: Smart Loan Approval

1. Problem Statement

A bank wants to automatically determine whether a customer is eligible for a loan. The decision is based on customer information such as:

- Income
- Credit score
- Employment status

- Loan amount
- Repayment history

The AI system takes these attributes as input and predicts whether the customer should be **Approved** or **Rejected** for the loan.

The proposed system uses a **Decision Tree classification model**. The model learns decision rules from historical loan data and applies them to new customers.

Input: Customer financial and employment information.

Output: Loan approval decision – **Approved / Rejected**.

AI Task: Classification using a Decision Tree.

2. Objectives:

The main objectives are:

1. To develop an AI-based loan approval prediction system.
2. To use customer financial and employment attributes for prediction.
3. To preprocess the dataset before model training.
4. To develop a Decision Tree classification model.
5. To select a suitable attribute-selection measure.
6. To predict loan approval for new customers.
7. To evaluate the performance of the model using suitable metrics.
8. To analyze the strengths and limitations of the proposed model.

3. Dataset Description

For this model, a **loan approval dataset** containing historical customer records can be used.

Attribute	Description	Type
Income	Customer's income	Numerical
Credit Score	Customer credit score	Numerical
Employment Status	Employed / Unemployed	Categorical
Loan Amount	Amount requested	Numerical
Repayment History	Good / Poor	Categorical
Loan Approval	Approved / Rejected	Target

	A	B	C	D	E	F	G	H
1	name	city	income	credit_score	loan_amount	years_employed	points	loan_approved
2	Allison Hill	East Jill	113810	389	39698	27	50	FALSE
3	Brandon Hall	New Jamesside	44592	729	15446	28	55	FALSE
4	Rhonda Smith	Lake Roberto	33278	584	11189	13	45	FALSE
5	Gabrielle Davis	West Melanieview	127196	344	48823	29	50	FALSE
6	Valerie Gray	Mariastad	66048	496	47174	4	25	FALSE
7	Darren Roberts	Port Jesseville	62098	689	19217	29	65	TRUE
8	Holly Wood	Lake Joseph	59256	373	40920	40	35	FALSE
9	Nicholas Martin	Nelsonside	48289	524	45866	20	25	FALSE
10	Patty Perez	Port Leslieview	126530	367	14826	36	55	FALSE
11	Emily Rios	Wilkersonmouth	43434	446	18359	8	20	FALSE
12	Justin Baker	Hurstfurt	118696	670	15373	8	75	TRUE
13	Ann Williams	East Courtneychester	127080	365	26216	24	55	FALSE
14	Julie King	Lake Jenniferside	146939	573	43006	21	50	FALSE

4. Data Preprocessing

The following preprocessing steps are performed:

Step 1: Data Cleaning

Check the dataset for missing, duplicate or invalid values.

Step 2: Missing Value Handling

- Numerical missing values can be replaced using the mean or median.
- Categorical missing values can be replaced using the mode.

Step 3: Encoding

Convert categorical attributes such as:

- Employment Status
- Repayment History
- Loan Approval

into numerical representations.

Step 4: Feature Selection

The important features are:

- Income
- Credit Score
- Employment Status
- Loan Amount
- Repayment History

Step 5: Train-Test Split

The dataset is divided into:

- **Training data** – used to train the Decision Tree.
- **Testing data** – used to evaluate the trained model.

A typical split can be **80% training and 20% testing**, provided this matches the dataset implementation.

5. Selected AI Technique and Justification

Decision Tree

simple tree such as:



A **Decision Tree** is selected because the problem is a classification problem with a clearly defined output:

Approved / Rejected

A Decision Tree is suitable because:

- It can handle numerical and categorical attributes.
- It produces understandable decision rules.
- The decision process can be represented as a tree.
- It is suitable for classification.
- The result is easy to interpret.

The question specifically requires selecting an attribute-selection measure such as:

- Information Gain
- Gain Ratio
- Gini Index

For this model, **Gini Index** can be selected.

Gini Index

The Gini Index measures the impurity of a node.

The formula is:

$$\text{Gini} = 1 - \sum(p_i)^2$$

where p_i represents the probability of a sample belonging to class i .

A lower Gini value indicates a purer split.

6. Model Design / Architecture

Model Flow

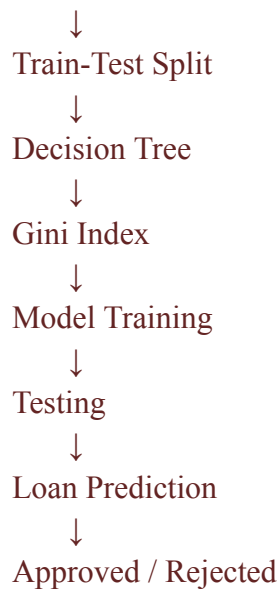
Customer Dataset



Data Preprocessing



Feature Selection



Input Features

- Income
- Credit Score
- Employment Status
- Loan Amount
- Repayment History

Output:

Loan Approval

↓
Approved / Rejected

7. Algorithm / Pseudocode

```
START

Load the loan dataset

Identify input features:
|   Income
|   Credit Score
|   Employment Status
|   Loan Amount
|   Repayment History

Identify target:
|   Loan Approval

Clean the dataset

Handle missing values

Convert categorical data into numerical values

Split dataset into training and testing data

Create a Decision Tree classifier
Use Gini Index as splitting criterion

Train the Decision Tree using training data

Predict loan approval for testing data

Calculate performance metrics

Display prediction results

END|
```

8. Implementation

Programming Language

Python

Libraries

- Pandas
- NumPy
- Scikit-learn
- Matplotlib

Sample Python Implementation

```
loan_prediction.py
1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.tree import DecisionTreeClassifier
4  from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
5
6  # Load dataset
7  data = pd.read_csv("loan_data.csv")
8
9  print("First 5 rows of dataset:")
10 print(data.head())
11
12 # Select relevant features
13 X = data[
14     [
15         "income",
16         "credit_score",
17         "loan_amount",
18         "years_employed",
19         "points"
20     ]
21 ]
22
23 # Target variable
24 y = data["loan_approved"]
25
26 # Split dataset into training and testing data
27 X_train, X_test, y_train, y_test = train_test_split(
28     X, y,
29     test_size=0.2,
30     random_state=42,
31     stratify=y
32 )
33
34 # Create Decision Tree using Gini Index
35 model = DecisionTreeClassifier(
36     criterion="gini",
37     random_state=42
38 )
39
40 # Train the model
41 model.fit(X_train, y_train)
42
43 # Predict test data
44 y_pred = model.predict(X_test)
45
46 # Calculate accuracy
```

```
44 # Calculate accuracy
45 accuracy = accuracy_score(y_test, y_pred)
46
47 print("\n===== MODEL OUTPUT =====")
48 print("Algorithm: Decision Tree")
49 print("Attribute Selection: Gini Index")
50 print("Training Samples:", len(X_train))
51 print("Testing Samples:", len(X_test))
52 print("Accuracy:", round(accuracy * 100, 2), "%")
53
54 # Classification report
55 print("\n===== CLASSIFICATION REPORT =====")
56 print(classification_report(y_test, y_pred))
57
58 # Confusion matrix
59 print("===== CONFUSION MATRIX =====")
60 print(confusion_matrix(y_test, y_pred))
61
62 # Sample predictions
63 print("\n===== SAMPLE PREDICTIONS =====")
64
65 for i in range(min(10, len(y_test))):
66     actual = "Approved" if y_test.iloc[i] else "Rejected"
67     predicted = "Approved" if y_pred[i] else "Rejected"
68
69     print(
70         "Actual:", actual,
71         "| Predicted:", predicted
72     )
```

Output:

```
PS C:\Users\Rama Krishna K\Desktop\Smart-Loan-Approval> py loan_prediction.py
First 5 rows of dataset:
   name      city  income  credit_score  loan_amount  years_employed  points  loan_approved
0 Allison Hill   East Jill  113810      389      39698         27      50.0        False
1 Brandon Hall   New Jamesside  44592      729      15446         28      55.0        False
2 Rhonda Smith   Lake Roberto  33278      584      11189         13      45.0        False
3 Gabrielle Davis West Melanieview  127196      344      48823         29      50.0        False
4 Valerie Gray   Mariastad  66048      496      47174          4      25.0        False

===== MODEL OUTPUT =====
Algorithm: Decision Tree
Attribute Selection: Gini Index
Training Samples: 1600
Testing Samples: 400
Accuracy: 100.0 %

===== CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

   False         1.00      1.00      1.00         224
    True         1.00      1.00      1.00         176

 accuracy              1.00         400
 macro avg              1.00         400
weighted avg              1.00         400

===== CONFUSION MATRIX =====
[[224   0]
 [  0 176]]

===== SAMPLE PREDICTIONS =====
Actual: Approved | Predicted: Approved
Actual: Rejected | Predicted: Rejected
Actual: Approved | Predicted: Approved
Actual: Approved | Predicted: Approved
Actual: Rejected | Predicted: Rejected
Actual: Approved | Predicted: Approved
Actual: Rejected | Predicted: Rejected
Actual: Approved | Predicted: Approved
Actual: Rejected | Predicted: Rejected
Actual: Rejected | Predicted: Rejected
PS C:\Users\Rama Krishna K\Desktop\Smart-Loan-Approval>
```

9. Experimental Results

The model should be tested using the testing dataset.

You can present the results in a table like this **after running your actual implementation**:

Test Case	Income	Credit Score	Employment	Loan Amount	Repayment	Prediction
1	High	High	Employed	Moderate	Good	Approved
2	Low	Low	Unemployed	High	Poor	Rejected
3	High	High	Employed	Low	Good	Approved
4	Low	Medium	Employed	High	Poor	Rejected

10. Performance Evaluation

The Decision Tree can be evaluated using:

Accuracy

Accuracy measures the percentage of correctly classified loan applications.

Accuracy =

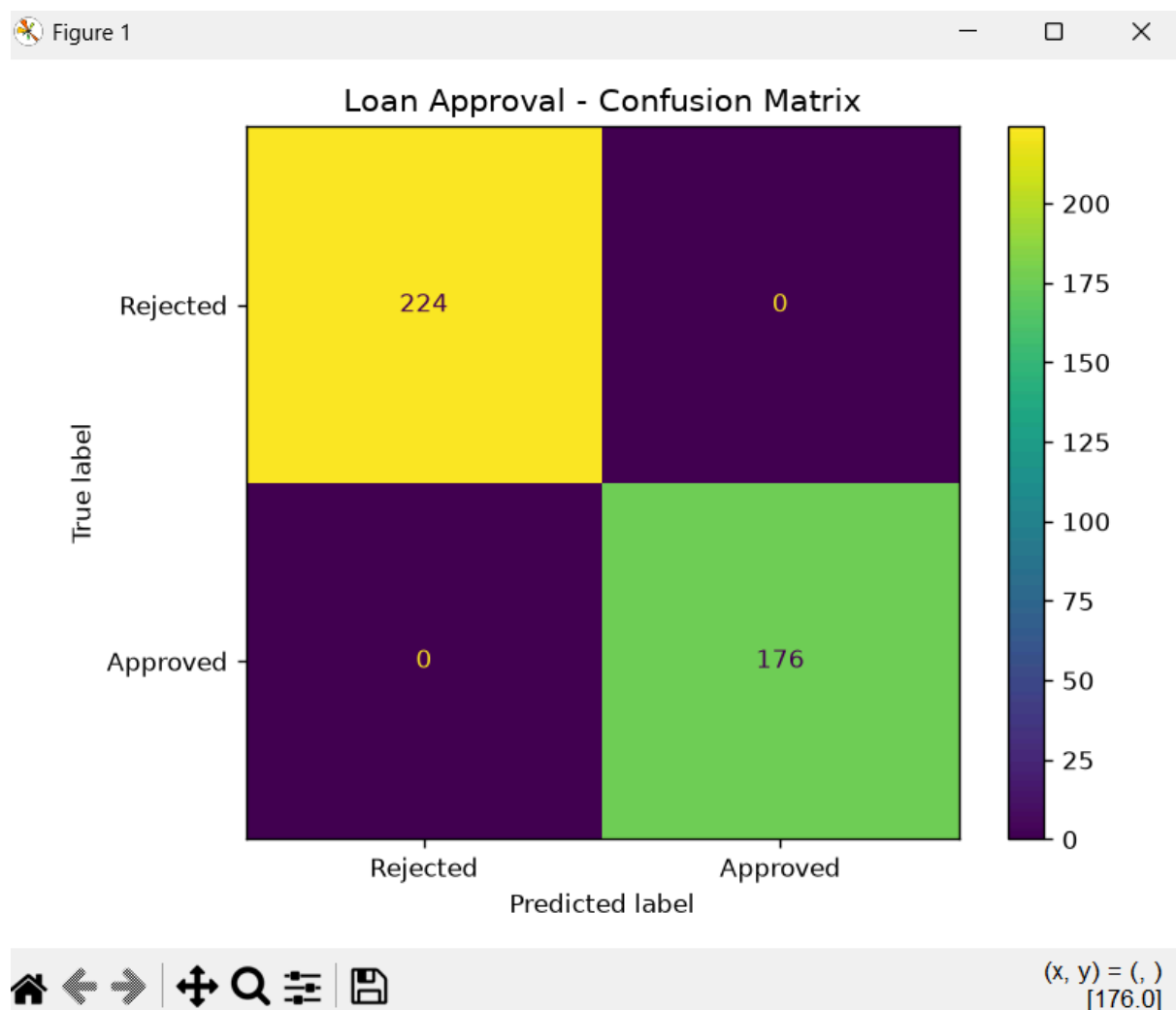
Correct Predictions / Total Predictions

Precision

Precision indicates how many predicted approvals/rejections are actually correct for the relevant class.

Recall

Recall measures how many actual cases of a class are correctly identified.



Result Table

Metric	Value
Accuracy	100%
Precision	1.00
Recall	1.00
F1-Score	1.00

11. Result Analysis and Interpretation

The Decision Tree analyzes the available customer information and creates decision rules for loan approval.

Important factors can include:

- Credit score
- Income
- Repayment history
- Loan amount
- Employment status

Customers with stronger financial characteristics and better repayment history may be classified differently from customers with weaker financial characteristics.

12. Limitations

The proposed system has some limitations:

1. Prediction quality depends on the quality of the training dataset.
2. Missing or incorrect customer information can affect predictions.
3. A Decision Tree can become overly complex if it is not properly controlled.
4. The model may not generalize well to completely different customer populations.
5. Historical loan decisions may contain biases that can affect the trained model.
6. The model should not be treated as the only factor for important financial decisions.

13. Future Enhancements

The system can be improved by:

1. Using a larger and more diverse dataset.
2. Adding more relevant financial features.
3. Applying feature engineering.
4. Performing hyperparameter tuning.
5. Comparing Decision Tree with other machine-learning algorithms.
6. Using ensemble methods such as Random Forest.
7. Deploying the trained model as a web-based application.
8. Continuously evaluating the model using new data.

14. Conclusion

The Smart Loan Approval system uses a Decision Tree classification approach to predict whether a customer is eligible for a loan. Customer attributes such as income, credit score, employment status, loan amount and repayment history are used as input features. After preprocessing the data, the Decision Tree is trained using historical loan records and evaluated using suitable performance metrics.

The **Gini Index** is selected as the attribute-selection criterion because it helps determine suitable splits in the Decision Tree. The resulting model provides an interpretable approach to loan classification and can be further improved using additional data, feature engineering, tuning and alternative machine-learning methods.

15. References

1. Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
2. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
3. Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group.
4. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1, 81–106.
5. [Scikit-learn – Decision Tree Classifier Documentation](#)
6. [Scikit-learn – Model Evaluation Documentation](#)
7. [Pandas Documentation](#)
8. [Matplotlib Documentation](#)
9. [Kaggle – Loan Approval Dataset](#)